

24. live 自動校正ノード

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 class、独自クラス、インスタンス、self、継承	7
6.7 先頭アンダースコア、__init__、名前付けの作法	7
6.8 rclpy、rcl、rmw、DDS/RTPS の層	7
6.9 ROS graph、Node、topic、service、action、parameter	8
6.10 callback、timer、Executor、spin、Callback Group	8
6.11 rqt_graph、ros2 CLI、rosviz をいつ使うか	9
6.12 制御、状態、入力、モデル予測制御 MPC	9
7 関数・クラスを上から順に解説	10
7.1 L13 関数 _clamp	10

7.2	L17 クラス SolarAutocalNode	10
7.3	L18 関数 SolarAutocalNode.__init__	12
7.4	L74 関数 SolarAutocalNode._on_speed	14
7.5	L77 関数 SolarAutocalNode._on_current	15
7.6	L82 関数 SolarAutocalNode._on_voltage	16
7.7	L87 関数 SolarAutocalNode._on_solar	16
7.8	L90 関数 SolarAutocalNode._on_env	17
7.9	L95 関数 SolarAutocalNode._on_metrics	18
7.10	L102 関数 SolarAutocalNode._ema	19
7.11	L106 関数 SolarAutocalNode._step	19
7.12	L153 関数 main	21
7.13	設定値と入力パラメータ	22
7.14	ROS topic I/O	22
8	処理の流れ	23
9	このファイルの位置づけ	23

1 対象

項目	内容
ファイル	mpc_solarcar/solar_autocal_node.py
ソース SHA-256	a98336e789670e85549bcdabb19c1a90b49d293dba6f193a6d91c053ef9abc51
種別	Python
区分	runtime node
役割	観測 solar/pack power と planner 予測との差から solar_gain、drive_power_gain、aux_power_w を推定 publish する。
起動文脈	運用中に mpc_node の係数を微調整する補助ノード。

2 このファイルを読む理由

観測 solar/pack power と planner 予測との差から solar_gain、drive_power_gain、aux_power_w を推定 publish する。

- /calib/* topic を publish する。
- mpc_node がそれを購読して内部 gain を更新する。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_solarcar/live_launch.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/solar_autocal_logic.py
- mpc_solarcar/mpc_node.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/solar_autocal_logic.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
return max(float(lo), min(float(hi), float(value)))

class SolarAutocalNode(Node):
    def __init__(self) -> None:
        super().__init__("solar_autocal_node")
        self.declare_parameter("publish_period_sec", 30.0)
```

```

self.declare_parameter("stationary_speed_kmh", 2.0)
self.declare_parameter("drive_speed_kmh", 25.0)
self.declare_parameter("day_ghi_threshold", 150.0)
self.declare_parameter("alpha", 0.2)
self.declare_parameter("solar_gain_init", 1.0)
self.declare_parameter("drive_power_gain_init", 1.0)
self.declare_parameter("aux_power_w_init", 8.0)
self.declare_parameter("solar_gain_min", 0.5)
self.declare_parameter("solar_gain_max", 1.5)
self.declare_parameter("drive_power_gain_min", 0.7)
self.declare_parameter("drive_power_gain_max", 1.4)
self.declare_parameter("aux_power_w_min", 0.0)
self.declare_parameter("aux_power_w_max", 300.0)

self.period = max(1.0, float(self.get_parameter("publish_period_sec").value))

```

5 主要構造の読み取り

主要クラスは SolarAutocalNode。主要関数は main。ROS パラメータ宣言は 14 件。ROS I/O は publisher 4 件、subscription 6 件。

5.1 主要クラス・関数

行	種別	名前	補足
17	class	SolarAutocalNode	bases: Node
13	function	_clamp	args: value, lo, hi
18	function	__init__	args: self
74	function	_on_speed	args: self, msg
77	function	_on_current	args: self, msg
82	function	_on_voltage	args: self, msg
87	function	_on_solar	args: self, msg
90	function	_on_env	args: self, msg
95	function	_on_metrics	args: self, msg
102	function	_ema	args: self, current, estimate, lo, hi
106	function	_step	args: self
153	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
13	関数 _clamp を定義する。
17	クラス SolarAutocalNode を定義する。
153	関数 main を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
---	----------	-------------

1	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>importmath</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L50, L51, L52, L53, L54, L55, L69, L70, ...。
5	<code>importrcclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L154, L156, L158。
6	<code>fromrcclpy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L17。
8	<code>fromstd_msgs. msgimportFloat32, Float32MultiArray,String</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L57, L58, L59, L60, L62, L63, L64, L65, ...。
10	<code>from.solar_autocal_ logicimportdaytime_ stationary_aux_estimate</code>	自動校正ロジック関数群から daytime_stationary_aux_estimate を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/solar_autocal_logic.py。このファイル内での主な使用位置は L110。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのものの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や ‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self.step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 rclpy、rcl、rmw、DDS/RTPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscriptionなどを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTSPS実装 -> ネットワークまたは同一PC
内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTSPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executor の実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.9 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.10 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は Multi-ThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.11 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosvbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_kmh /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.12 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L13 関数 `_clamp`

- 行範囲: L13–L14
- 所属: module 直下
- 定義: `_clamp(value:float, lo:float, hi:float) -> float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `max`, `min`
- 戻り値の要点: `max(float(lo), min(float(hi), float(value)))`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `max(float(lo), min(float(hi), float(value)))` を返す。

代表コード断片

```
def _clamp(value: float, lo: float, hi: float) -> float:
    return max(float(lo), min(float(hi), float(value)))
```

7.2 L17 クラス `SolarAutocalNode`

- 行範囲: L17–L150
- 所属: module 直下
- 定義: `SolarAutocalNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `_on_speed` を定義する。
3. 関数 `_on_current` を定義する。
4. 関数 `_on_voltage` を定義する。
5. 関数 `_on_solar` を定義する。
6. 関数 `_on_env` を定義する。
7. 関数 `_on_metrics` を定義する。
8. 関数 `_ema` を定義する。
9. 関数 `_step` を定義する。

代表コード断片

```
class SolarAutocalNode(Node):
    def __init__(self) -> None:
        super().__init__("solar_autocal_node")
        self.declare_parameter("publish_period_sec", 30.0)
        self.declare_parameter("stationary_speed_kmh", 2.0)
        self.declare_parameter("drive_speed_kmh", 25.0)
        self.declare_parameter("day_ghi_threshold", 150.0)
        self.declare_parameter("alpha", 0.2)
        self.declare_parameter("solar_gain_init", 1.0)
        self.declare_parameter("drive_power_gain_init", 1.0)
        self.declare_parameter("aux_power_w_init", 8.0)
        self.declare_parameter("solar_gain_min", 0.5)
        self.declare_parameter("solar_gain_max", 1.5)
        self.declare_parameter("drive_power_gain_min", 0.7)
        self.declare_parameter("drive_power_gain_max", 1.4)
        self.declare_parameter("aux_power_w_min", 0.0)
        self.declare_parameter("aux_power_w_max", 300.0)

        self.period = max(1.0, float(self.get_parameter("publish_period_sec").value))
        self.stationary_speed = float(self.get_parameter("stationary_speed_kmh").value)
        self.drive_speed = float(self.get_parameter("drive_speed_kmh").value)
        self.day_ghi = float(self.get_parameter("day_ghi_threshold").value)
        self.alpha = _clamp(float(self.get_parameter("alpha").value), 0.01, 1.0)
        self.solar_gain = float(self.get_parameter("solar_gain_init").value)
        self.drive_gain = float(self.get_parameter("drive_power_gain_init").value)
        self.aux_power = float(self.get_parameter("aux_power_w_init").value)
        self.solar_gain_min = float(self.get_parameter("solar_gain_min").value)
        self.solar_gain_max = float(self.get_parameter("solar_gain_max").value)
        self.drive_gain_min = float(self.get_parameter("drive_power_gain_min").value)
        self.drive_gain_max = float(self.get_parameter("drive_power_gain_max").value)
        self.aux_min = float(self.get_parameter("aux_power_w_min").value)
        self.aux_max = float(self.get_parameter("aux_power_w_max").value)

        self.speed_kmh = math.nan
        self.pack_power_w = math.nan
    ...
```

7.3 L18 関数 SolarAutocalNode.__init__

- 行範囲: L18–L72
- 所属: SolarAutocalNode
- 定義: `__init__(self)->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `__init__, _clamp, create_publisher, create_subscription, create_timer, declare_parameter, float, get_logger, get_parameter, info, max, super`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._on_current, self._on_env, self._on_metrics, self._on_solar, self._on_speed, self._on_voltage, self._step, self.create_publisher, self.create_subscription, self.create_timer, self.declare_parameter, self.get_logger, self.get_parameter, self.period`
- 更新する主なインスタンス属性: `self._latest_current_a, self._latest_voltage_v, self.alpha, self.aux_max, self.aux_min, self.aux_power, self.day_ghi, self.drive_gain, self.drive_gain_max, self.drive_gain_min, self.drive_speed, self.ghi, self.pack_power_w, self.period, self.pred_pack_w, self.pred_solar_w, self.pub_aux, self.pub_drive, self.pub_solar, self.pub_status, self.solar_gain, self.solar_gain_max, self.solar_gain_min, self.solar_obs_w`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。
8. `self.declare_parameter(...)` を実行する。
9. `self.declare_parameter(...)` を実行する。
10. `self.declare_parameter(...)` を実行する。
11. `self.declare_parameter(...)` を実行する。
12. `self.declare_parameter(...)` を実行する。

13. `self.declare_parameter(...)` を実行する。
14. `self.declare_parameter(...)` を実行する。
15. `self.declare_parameter(...)` を実行する。
16. `self.period` に `max(1.0, float(self.get_parameter('publish_period_sec').value))` の結果を代入する。
17. `self.stationary_speed` に `float(self.get_parameter('stationary_speed_kmh').value)` の結果を代入する。
18. `self.drive_speed` に `float(self.get_parameter('drive_speed_kmh').value)` の結果を代入する。
19. `self.day_ghi` に `float(self.get_parameter('day_ghi_threshold').value)` の結果を代入する。
20. `self.alpha` に `_clamp(float(self.get_parameter('alpha').value), 0.01, 1.0)` の結果を代入する。
21. `self.solar_gain` に `float(self.get_parameter('solar_gain_init').value)` の結果を代入する。
22. `self.drive_gain` に `float(self.get_parameter('drive_power_gain_init').value)` の結果を代入する。
23. `self.aux_power` に `float(self.get_parameter('aux_power_w_init').value)` の結果を代入する。
24. `self.solar_gain_min` に `float(self.get_parameter('solar_gain_min').value)` の結果を代入する。
25. `self.solar_gain_max` に `float(self.get_parameter('solar_gain_max').value)` の結果を代入する。
26. `self.drive_gain_min` に `float(self.get_parameter('drive_power_gain_min').value)` の結果を代入する。
27. `self.drive_gain_max` に `float(self.get_parameter('drive_power_gain_max').value)` の結果を代入する。
28. `self.aux_min` に `float(self.get_parameter('aux_power_w_min').value)` の結果を代入する。
29. `self.aux_max` に `float(self.get_parameter('aux_power_w_max').value)` の結果を代入する。
30. `self.speed_kmh` に `math.nan` の結果を代入する。
31. `self.pack_power_w` に `math.nan` の結果を代入する。
32. `self.solar_obs_w` に `math.nan` の結果を代入する。
33. `self.ghi` に `math.nan` の結果を代入する。
34. `self.pred_solar_w` に `math.nan` の結果を代入する。
35. `self.pred_pack_w` に `math.nan` の結果を代入する。
36. `self.pub_solar` に `self.create_publisher(Float32, '/calib/solar_gain', 10)` の結果を代入する。
37. `self.pub_drive` に `self.create_publisher(Float32, '/calib/drive_power_gain', 10)` の結果を代入する。
38. `self.pub_aux` に `self.create_publisher(Float32, '/calib/aux_power_w', 10)` の結果を代入する。
39. `self.pub_status` に `self.create_publisher(String, '/calib/status', 10)` の結果を代入する。
40. `self.create_subscription(...)` を実行する。
41. `self.create_subscription(...)` を実行する。
42. `self.create_subscription(...)` を実行する。
43. `self.create_subscription(...)` を実行する。
44. `self.create_subscription(...)` を実行する。
45. `self.create_subscription(...)` を実行する。
46. `self._latest_current_a` に `math.nan` の結果を代入する。

47. self._latest_voltage_v に math.nan の結果を代入する。
48. self.timer に self.create_timer(self.period, self._step) の結果を代入する。
49. self.get_logger().info(...) を実行する。

代表コード断片

```
def __init__(self) -> None:
    super().__init__("solar_autocal_node")
    self.declare_parameter("publish_period_sec", 30.0)
    self.declare_parameter("stationary_speed_kmh", 2.0)
    self.declare_parameter("drive_speed_kmh", 25.0)
    self.declare_parameter("day_ghi_threshold", 150.0)
    self.declare_parameter("alpha", 0.2)
    self.declare_parameter("solar_gain_init", 1.0)
    self.declare_parameter("drive_power_gain_init", 1.0)
    self.declare_parameter("aux_power_w_init", 8.0)
    self.declare_parameter("solar_gain_min", 0.5)
    self.declare_parameter("solar_gain_max", 1.5)
    self.declare_parameter("drive_power_gain_min", 0.7)
    self.declare_parameter("drive_power_gain_max", 1.4)
    self.declare_parameter("aux_power_w_min", 0.0)
    self.declare_parameter("aux_power_w_max", 300.0)

    self.period = max(1.0, float(self.get_parameter("publish_period_sec").value))
    self.stationary_speed = float(self.get_parameter("stationary_speed_kmh").value)
    self.drive_speed = float(self.get_parameter("drive_speed_kmh").value)
    self.day_ghi = float(self.get_parameter("day_ghi_threshold").value)
    self.alpha = _clamp(float(self.get_parameter("alpha").value), 0.01, 1.0)
    self.solar_gain = float(self.get_parameter("solar_gain_init").value)
    self.drive_gain = float(self.get_parameter("drive_power_gain_init").value)
    self.aux_power = float(self.get_parameter("aux_power_w_init").value)
    self.solar_gain_min = float(self.get_parameter("solar_gain_min").value)
    self.solar_gain_max = float(self.get_parameter("solar_gain_max").value)
    self.drive_gain_min = float(self.get_parameter("drive_power_gain_min").value)
    self.drive_gain_max = float(self.get_parameter("drive_power_gain_max").value)
    self.aux_min = float(self.get_parameter("aux_power_w_min").value)
    self.aux_max = float(self.get_parameter("aux_power_w_max").value)

    self.speed_kmh = math.nan
    self.pack_power_w = math.nan
    self.solar_obs_w = math.nan
    ...
```

7.4 L74 関数 SolarAutocalNode._on_speed

- 行範囲: L74–L75
- 所属: SolarAutocalNode
- 定義: _on_speed(self, msg: Float32) -> None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: `self.speed_kmh`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self.speed_kmh` に `float(msg.data)` の結果を代入する。

代表コード断片

```
def _on_speed(self, msg: Float32) -> None:
    self.speed_kmh = float(msg.data)
```

7.5 L77 関数 `SolarAutocalNode._on_current`

- 行範囲: L77-L80
- 所属: `SolarAutocalNode`
- 定義: `_on_current(self, msg: Float32) -> None`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._latest_current_a`, `self._latest_voltage_v`
- 更新する主なインスタンス属性: `self._latest_current_a`, `self.pack_power_w`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self._latest_current_a` に `float(msg.data)` の結果を代入する。
2. 条件 `math.isfinite(self._latest_voltage_v)` を判定し、真なら内部処理を行う。
3. `self.pack_power_w` に `self._latest_current_a * self._latest_voltage_v` の結果を代入する。

代表コード断片

```
def _on_current(self, msg: Float32) -> None:
    self._latest_current_a = float(msg.data)
    if math.isfinite(self._latest_voltage_v):
        self.pack_power_w = self._latest_current_a * self._latest_voltage_v
```

7.6 L82 関数 SolarAutocalNode._on_voltage

- 行範囲: L82–L85
- 所属: SolarAutocalNode
- 定義: `_on_voltage(self, msg: Float32) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._latest_current_a`, `self._latest_voltage_v`
- 更新する主なインスタンス属性: `self._latest_voltage_v`, `self.pack_power_w`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self._latest_voltage_v` に `float(msg.data)` の結果を代入する。
2. 条件 `math.isfinite(self._latest_current_a)` を判定し、真なら内部処理を行う。
3. `self.pack_power_w` に `self._latest_current_a * self._latest_voltage_v` の結果を代入する。

代表コード断片

```
def _on_voltage(self, msg: Float32) -> None:
    self._latest_voltage_v = float(msg.data)
    if math.isfinite(self._latest_current_a):
        self.pack_power_w = self._latest_current_a * self._latest_voltage_v
```

7.7 L87 関数 SolarAutocalNode._on_solar

- 行範囲: L87–L88
- 所属: SolarAutocalNode
- 定義: `_on_solar(self, msg: Float32) -> None`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: float
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: self.solar_obs_w
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. self.solar_obs_w に float(msg.data) の結果を代入する。

代表コード断片

```
def _on_solar(self, msg: Float32) -> None:
    self.solar_obs_w = float(msg.data)
```

7.8 L90 関数 SolarAutocalNode._on_env

- 行範囲: L90–L93
- 所属: SolarAutocalNode
- 定義: _on_env(self, msg: Float32MultiArray) -> None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float, len, list
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: data
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: self.ghi
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. data に list(msg.data) の結果を代入する。
2. 条件 len(data) >= 1 を判定し、真なら内部処理を行う。
3. self.ghi に float(data[0]) の結果を代入する。

代表コード断片

```
def _on_env(self, msg: Float32MultiArray) -> None:
    data = list(msg.data)
    if len(data) >= 1:
        self.ghi = float(data[0])
```

7.9 L95 関数 SolarAutocalNode._on_metrics

- 行範囲: L95–L100
- 所属: SolarAutocalNode
- 定義: _on_metrics(self, msg: Float32MultiArray) -> None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float, len, list
- 戻り値の要点: 戻り値が無いが、return 文が明示されていない。
- 主なローカル代入名: data
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: self.pred_pack_w, self.pred_solar_w
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. data に list(msg.data) の結果を代入する。
2. 条件 len(data) >= 6 を判定し、真なら内部処理を行う。
3. self.pred_solar_w に float(data[5]) の結果を代入する。
4. 条件 len(data) >= 9 を判定し、真なら内部処理を行う。
5. self.pred_pack_w に float(data[8]) の結果を代入する。

代表コード断片

```
def _on_metrics(self, msg: Float32MultiArray) -> None:
    data = list(msg.data)
    if len(data) >= 6:
        self.pred_solar_w = float(data[5])
    if len(data) >= 9:
        self.pred_pack_w = float(data[8])
```

7.10 L102 関数 SolarAutocalNode._ema

- 行範囲: L102–L104
- 所属: SolarAutocalNode
- 定義: `_ema(self, current:float, estimate:float, lo:float, hi:float)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_clamp`, `float`
- 戻り値の要点: `_clamp(blended, lo, hi)`
- 主なローカル代入名: `blended`
- 読み取る主なインスタンス属性: `self.alpha`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `blended` に `(1.0 - self.alpha) * float(current) + self.alpha * float(estimate)` の結果を代入する。
2. `_clamp(blended, lo, hi)` を返す。

代表コード断片

```
def _ema(self, current: float, estimate: float, lo: float, hi: float) -> float:
    blended = (1.0 - self.alpha) * float(current) + self.alpha * float(estimate)
    return _clamp(blended, lo, hi)
```

7.11 L106 関数 SolarAutocalNode._step

- 行範囲: L106–L150
- 所属: SolarAutocalNode
- 定義: `_step(self)->None`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: Float32, String, _ema, abs, daytime_stationary_aux_estimate, float, isfinite, max, publish
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: aux_estimate, gain_est, status
- 読み取る主なインスタンス属性: self._ema, self.aux_max, self.aux_min, self.aux_power, self.day_ghi, self.drive_gain, self.drive_gain_max, self.drive_gain_min, self.drive_speed, self.ghi, self.pack_power_w, self.pred_pack_w, self.pred_solar_w, self.pub_aux, self.pub_drive, self.pub_solar, self.pub_status, self.solar_gain, self.solar_gain_max, self.solar_gain_min, self.solar_obs_w, self.speed_kmh, self.stationary_speed
- 更新する主なインスタンス属性: self.aux_power, self.drive_gain, self.solar_gain
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. aux_estimate に daytime_stationary_aux_estimate(ghi_wm2=self.ghi, day_ghi_threshold_wm2=self.day_ghi, speed_kmh=self.speed_kmh, stationary_speed_kmh=self.stationary_speed, pack_power_w=self.pack_power_w, solar_power_w=self.solar_obs_w) の結果を代入する。
2. 条件 aux_estimate is not None を判定し、真なら内部処理を行う。
3. self.aux_power に self._ema(self.aux_power, aux_estimate, self.aux_min, self.aux_max) の結果を代入する。
4. 条件 math.isfinite(self.ghi) and self.ghi >= self.day_ghi and math.isfinite(self.solar_obs_w) and math.isfinite(self.pred_solar_w) and (abs(self.pred_solar_w) >= 20.0) を判定し、真なら内部処理を行う。
5. gain_est に self.solar_obs_w / self.pred_solar_w の結果を代入する。
6. self.solar_gain に self._ema(self.solar_gain, gain_est, self.solar_gain_min, self.solar_gain_max) の結果を代入する。
7. 条件 math.isfinite(self.speed_kmh) and self.speed_kmh >= self.drive_speed and math.isfinite(self.pack_power_w) and math.isfinite(self.pred_pack_w) and (abs(self.pred_pack_w) >= 50.0) and (self.pack_power_w > 0.0) を判定し、真なら内部処理を行う。
8. gain_est に self.pack_power_w / max(1.0, self.pred_pack_w) の結果を代入する。
9. self.drive_gain に self._ema(self.drive_gain, gain_est, self.drive_gain_min, self.drive_gain_max) の結果を代入する。
10. self.pub_solar.publish(...) を実行する。
11. self.pub_drive.publish(...) を実行する。
12. self.pub_aux.publish(...) を実行する。
13. status に f'solar_gain={self.solar_gain:.3f} drive_power_gain={self.drive_gain:.3f} aux_power_w={self.aux_power:.1f}' の結果を代入する。
14. self.pub_status.publish(...) を実行する。

代表コード断片

```
def _step(self) -> None:
    # The team switches auxiliaries off at night, so a night stationary
    # sample is not an estimate of the daytime auxiliary load. At a
    # daytime stop  $P_{pack} = P_{aux} - P_{pv}$ , hence  $P_{aux} = P_{pack} + P_{pv}$ .
    aux_estimate = daytime_stationary_aux_estimate(
        ghi_wm2=self.ghi,
        day_ghi_threshold_wm2=self.day_ghi,
        speed_kmh=self.speed_kmh,
        stationary_speed_kmh=self.stationary_speed,
        pack_power_w=self.pack_power_w,
        solar_power_w=self.solar_obs_w,
    )
    if aux_estimate is not None:
        self.aux_power = self._ema(self.aux_power, aux_estimate, self.aux_min, self.
aux_max)

    if (
        math.isfinite(self.ghi)
        and self.ghi >= self.day_ghi
        and math.isfinite(self.solar_obs_w)
        and math.isfinite(self.pred_solar_w)
        and abs(self.pred_solar_w) >= 20.0
    ):
        gain_est = self.solar_obs_w / self.pred_solar_w
        self.solar_gain = self._ema(self.solar_gain, gain_est, self.solar_gain_min, self.
.solar_gain_max)

    if (
        math.isfinite(self.speed_kmh)
        and self.speed_kmh >= self.drive_speed
        and math.isfinite(self.pack_power_w)
        and math.isfinite(self.pred_pack_w)
        and abs(self.pred_pack_w) >= 50.0
        and self.pack_power_w > 0.0
    ):
        gain_est = self.pack_power_w / max(1.0, self.pred_pack_w)
        self.drive_gain = self._ema(self.drive_gain, gain_est, self.drive_gain_min, self.
.drive_gain_max)
    ...
```

7.12 L153 関数 main

- 行範囲: L153–L158
- 所属: module 直下
- 定義: `main()->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `SolarAutocalNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `node`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `rclpy.init(...)` を実行する。
2. `node` に `SolarAutocalNode()` の結果を代入する。
3. `rclpy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rclpy.shutdown(...)` を実行する。

代表コード断片

```
def main() -> None:
    rclpy.init()
    node = SolarAutocalNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

7.13 設定値と入力パラメータ

行	名前	既定値・補足
20	<code>publish_period_sec</code>	30.0
21	<code>stationary_speed_kmh</code>	2.0
22	<code>drive_speed_kmh</code>	25.0
23	<code>day_ghi_threshold</code>	150.0
24	<code>alpha</code>	0.2
25	<code>solar_gain_init</code>	1.0
26	<code>drive_power_gain_init</code>	1.0
27	<code>aux_power_w_init</code>	8.0
28	<code>solar_gain_min</code>	0.5
29	<code>solar_gain_max</code>	1.5
30	<code>drive_power_gain_min</code>	0.7
31	<code>drive_power_gain_max</code>	1.4
32	<code>aux_power_w_min</code>	0.0
33	<code>aux_power_w_max</code>	300.0

7.14 ROS topic I/O

行	種別	topic	callback / 補足
57	publisher	<code>/calib/solar_gain</code>	publisher
58	publisher	<code>/calib/drive_power_gain</code>	publisher
59	publisher	<code>/calib/aux_power_w</code>	publisher
60	publisher	<code>/calib/status</code>	publisher

62	subscription	/vehicle/speed_kmh	self._on_speed
63	subscription	/vehicle/batt_current_a	self._on_current
64	subscription	/vehicle/batt_voltage_v	self._on_voltage
65	subscription	/vehicle/solar_power_w	self._on_solar
66	subscription	/planner/env	self._on_env
67	subscription	/planner/metrics	self._on_metrics

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_autocal_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。